

EECE 7205: Fundamentals of Computer Engineering

Fall 2026

Assignment #01 Sorting-based Interview Problems

Topics: Sorting, Intervals, Hash Maps, Heaps, Merge Sort, Quickselect

Student Name	Lingtong.Ge
NUID	003917506
Email	ge.lingt@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	2026-09-22

Submission Checklist

- Submit **one PDF** compiled from this Overleaf report.
- Include **clear screenshots** for each problem (text must be readable).
- Include your **GitHub repository link** in the report.
- File name: `Assignment01_LastName.pdf`

How to complete this assignment

1. For each problem, complete the 7 steps: Understand, Design, Pseudocode, Correctness, Complexity, C++ Implementation, Test Cases.
2. Include screenshots showing the selected problem, input, and output.
3. Push the PDF and `cpp/` folder to your GitHub repository.

Problem 1.

1. Understand the Problem

Goal: Given a collection of intervals $[start, end]$, merge all overlapping or touching intervals and return a set of non-overlapping intervals.

Input: `vector<vector<int>>` where each element is an interval $[start, end]$.

Output: `vector<vector<int>>` of merged, non-overlapping intervals.

Constraints and Assumptions:

- $start \leq end$ for every interval.
- Two intervals that only touch at an endpoint (e.g., $[1, 3]$ and $[3, 5]$) are considered mergeable.
- Empty input returns an empty result.
- A single interval is returned as-is.

Edge Cases: empty input; single interval; all intervals disjoint; fully nested intervals; touching endpoints.

2. Design the Solution

Sort the intervals by their starting value, then perform a single linear scan to merge.

Why this fits: After sorting by start, any intervals that can overlap with the current one must appear adjacent to it. Therefore, one pass is enough to merge all overlaps, avoiding pairwise comparisons.

3. Pseudocode

```
1 function mergeIntervals(intervals):
2     if intervals is empty:
3         return empty list
4
5     sort intervals by start ascending
6
7     merged = empty list
8     for each interval in intervals:
9         if merged is empty or interval.start > merged.last.end:
10            merged.add(interval)
11        else:
12            merged.last.end = max(merged.last.end, interval.end)
13
14    return merged
```

4. Correctness

After sorting, if the current interval does not overlap the last interval in **merged**, it cannot overlap any earlier interval either, because earlier intervals end no later than the last one. So adding it directly is safe. If it does overlap (or touches), we extend the last interval's end to cover both. The condition uses strict $>$ so touching endpoints (e.g., $[1, 3]$ and $[3, 5]$) are still merged.

5. Complexity Analysis

- **Time:** $O(n \log n)$ for sorting, plus $O(n)$ for the scan. Overall $O(n \log n)$.
- **Space:** $O(n)$ for the result; extra space is $O(1)$ or $O(\log n)$ depending on the sort.

6. C++ Implementation

```
1 vector<vector<int>> mergeIntervals(vector<vector<int>> intervals) {
2     if (intervals.empty()) return {};
3
4     sort(intervals.begin(), intervals.end(),
5         [](const vector<int>& a, const vector<int>& b) {
6             return a[0] < b[0];
7         });
8
9     vector<vector<int>> merged;
10    for (const auto& interval : intervals) {
11        if (merged.empty() || interval[0] > merged.back()[1]) {
12            merged.push_back(interval);
```

```

13         } else {
14             merged.back()[1] = max(merged.back()[1], interval[1]);
15         }
16     }
17     return merged;
18 }

```

7. Test Cases

Example:

Input: `[[1,3],[2,6],[8,10],[15,18]]`

Output: `[[1,6],[8,10],[15,18]]`

Additional 1:

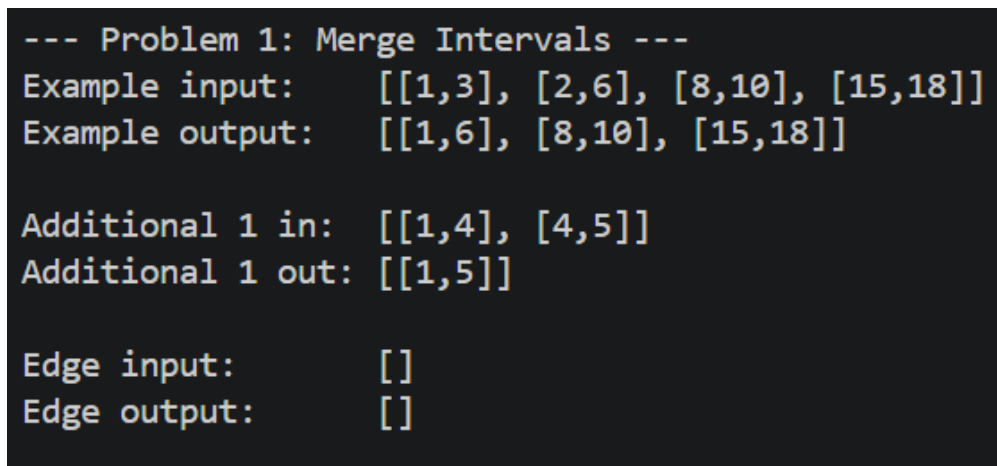
Input: `[[1,4],[4,5]]`

Output: `[[1,5]]` (touching endpoints merged)

Additional 2 (Edge):

Input: `[]`

Output: `[]`



```

--- Problem 1: Merge Intervals ---
Example input:      [[1,3], [2,6], [8,10], [15,18]]
Example output:     [[1,6], [8,10], [15,18]]

Additional 1 in:    [[1,4], [4,5]]
Additional 1 out:   [[1,5]]

Edge input:         []
Edge output:         []

```

Figure 1: Problem 1 execution screenshot

Problem 2.

1. Understand the Problem

Goal: Given an array containing only 0, 1, and 2, sort it in place so that all 0s come first, then 1s, then 2s.

Input: `vector<int>` with values in `{0, 1, 2}`.

Output: The same vector, sorted in place.

Constraints: Do not use `std::sort`. Implement two solutions: counting and the one-pass Dutch National Flag.

Edge Cases: empty array; all equal; already sorted; reverse sorted.

2. Design the Solution

Counting: Count occurrences of 0, 1, 2, then rewrite the array.

Dutch National Flag: Maintain three pointers `low`, `mid`, `high`. Elements before `low` are 0, between `low` and `mid` are 1, after `high` are 2.

3. Pseudocode

```
1 function sortColorsCounting(nums):
2     count0 = count1 = count2 = 0
3     for x in nums:
4         if x == 0: count0++
5         else if x == 1: count1++
6         else: count2++
7     index = 0
8     repeat count0 times: nums[index++] = 0
9     repeat count1 times: nums[index++] = 1
10    repeat count2 times: nums[index++] = 2
11
12 function sortColorsDutch(nums):
13     low = 0; mid = 0; high = nums.size() - 1
14     while mid <= high:
15         if nums[mid] == 0:
16             swap(nums[low], nums[mid]); low++; mid++
17         else if nums[mid] == 1:
18             mid++
19         else:
20             swap(nums[mid], nums[high]); high--
```

4. Correctness

Counting rewrites the array from frequencies, so it is trivially sorted. For Dutch Flag, the invariant is: $[0, low)$ are 0, $[low, mid)$ are 1, $(high, n - 1]$ are 2. Each step preserves the invariant, and the loop ends when $mid > high$, at which point the entire array is sorted.

5. Complexity Analysis

- Counting: $O(n)$ time, $O(1)$ space.
- Dutch Flag: $O(n)$ time, $O(1)$ space.

6. C++ Implementation

```
1 void sortColorsCounting(vector<int>& nums) {
2     int count0 = 0, count1 = 0, count2 = 0;
3     for (int x : nums) {
4         if (x == 0) count0++;
5         else if (x == 1) count1++;
6         else count2++;
7     }
8     int idx = 0;
9     while (count0--> 0) nums[idx++] = 0;
```

```

10     while (count1--> 0) nums[idx++] = 1;
11     while (count2--> 0) nums[idx++] = 2;
12 }
13
14 void sortColorsDutch(vector<int>& nums) {
15     int low = 0, mid = 0, high = (int)nums.size() - 1;
16     while (mid <= high) {
17         if (nums[mid] == 0) { swap(nums[low], nums[mid]); low++; mid++; }
18         else if (nums[mid] == 1) { mid++; }
19         else { swap(nums[mid], nums[high]); high--; }
20     }
21 }

```

7. Test Cases

Example: input [2,0,2,1,1,0], output [0,0,1,1,2,2]

Additional 1: input [2,2,2], output [2,2,2]

Edge: input [], output []

```

--- Problem 2: Sort Colors ---
Example input:      [2, 0, 2, 1, 1, 0]
Counting out:      [0, 0, 1, 1, 2, 2]
Dutch out:         [0, 0, 1, 1, 2, 2]

Additional 1 in:    [2, 2, 2]
Counting out:      [2, 2, 2]
Dutch out:         [2, 2, 2]

Edge input:        []
Counting out:      []
Dutch out:         []

```

Figure 2: Problem 2 execution screenshot

Problem 3.

1. Understand the Problem

Goal: Given a vector of non-negative integers, arrange them so that their concatenation forms the largest possible number.

Input: `vector<int>` of non-negative integers.

Output: A string representing the largest concatenation.

Edge Cases: empty vector; all zeros; mixed zeros and non-zeros; different digit lengths (9 vs 34).

2. Design the Solution

Convert each integer to a string and sort using comparator: a before b if $a + b > b + a$. Concatenate. If the result starts with 0, return "0".

3. Pseudocode

```
1 function largestNumber(nums):
2     if nums is empty: return "0"
3     strings = convert each num to string
4     sort strings with comparator:  $a + b > b + a$ 
5     result = concatenate all strings
6     if result[0] == '0': return "0"
7     return result
```

4. Correctness

If $a+b > b+a$, placing a before b yields a larger concatenation. Sorting by this comparator produces an order such that no adjacent swap increases the total, so the concatenation is maximal. Leading zeros only occur when all numbers are zero.

5. Complexity Analysis

- Time: $O(n \log n \cdot L)$, L = average string length.
- Space: $O(n \cdot L)$.

6. C++ Implementation

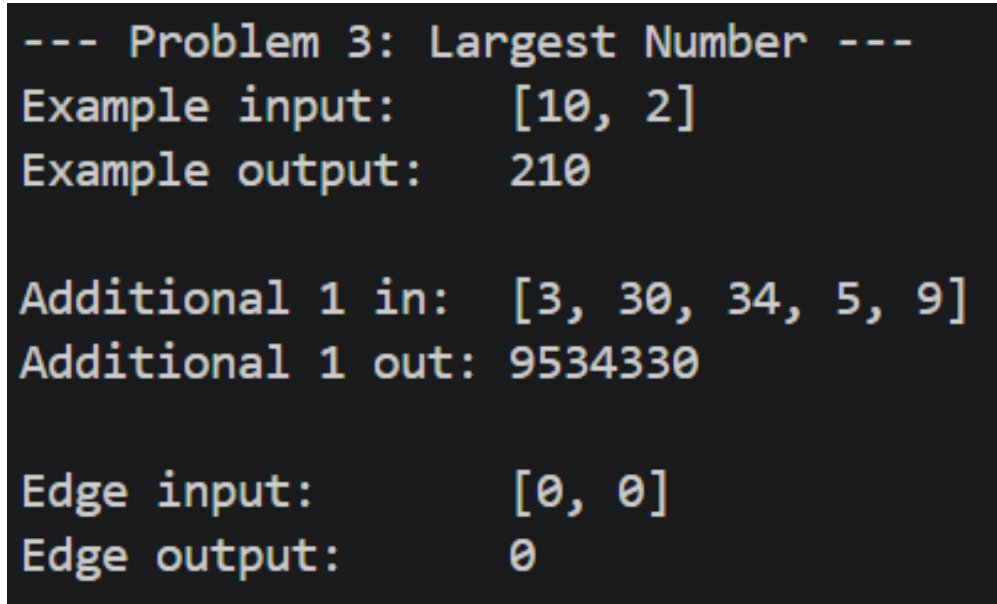
```
1 string largestNumber(vector<int> nums) {
2     if (nums.empty()) return "0";
3     vector<string> strs;
4     for (int x : nums) strs.push_back(to_string(x));
5     sort(strs.begin(), strs.end(), [](const string& a, const string& b) {
6         return a + b > b + a;
7     });
8     if (strs[0] == "0") return "0";
9     string result;
10    for (const string& s : strs) result += s;
11    return result;
12 }
```

7. Test Cases

Example: input [10,2], output "210"

Additional 1: input [3,30,34,5,9], output "9534330"

Edge: input [0,0], output "0"



```
--- Problem 3: Largest Number ---
Example input:    [10, 2]
Example output:   210

Additional 1 in:  [3, 30, 34, 5, 9]
Additional 1 out: 9534330

Edge input:      [0, 0]
Edge output:     0
```

Figure 3: Problem 3 execution screenshot

Problem 4.

1. Understand the Problem

Goal: Given a vector of strings, group anagrams together.

Input: vector<string>.

Output: vector<vector<string>> where each inner vector is a group. Group order does not matter; within each group, input order must be preserved.

Edge Cases: empty input; all identical; all distinct; empty string.

2. Design the Solution

Hash map keyed by the sorted characters of each string. Traverse input in order and append each string to its group.

3. Pseudocode

```
1 function groupAnagrams(strs):
2     map = empty hash map from string to list of strings
3     for s in strs:
4         key = sorted characters of s
```

```

5         map[key].add(s)
6     result = empty list
7     for each group in map:
8         result.add(group)
9     return result

```

4. Correctness

Two strings are anagrams iff their sorted characters are equal, so they land in the same bucket. Inserting in input order preserves original order within each group.

5. Complexity Analysis

- Time: $O(n \cdot L \log L)$.
- Space: $O(n \cdot L)$.

6. C++ Implementation

```

1 vector<vector<string>> groupAnagrams(vector<string> strs) {
2     unordered_map<string, vector<string>> map;
3     for (const string& s : strs) {
4         string key = s;
5         sort(key.begin(), key.end());
6         map[key].push_back(s);
7     }
8     vector<vector<string>> result;
9     for (auto& pair : map) result.push_back(pair.second);
10    return result;
11 }

```

7. Test Cases

Example: input ["eat","tea","tan","ate","nat","bat"], grouped output.

Additional 1: input [""], output [[[""]]].

Edge: input [], output [].

Problem 5.

1. Understand the Problem

Goal: Given meeting intervals $[start, end]$, determine whether one person can attend all. A meeting starting exactly when another ends is allowed.

Input: vector<vector<int>> of intervals.

Output: bool.

Edge Cases: empty input (true); single meeting (true); touching (true); overlapping (false).

```
--- Problem 4: Group Anagrams ---
Example input:    [eat, tea, tan, ate, nat, bat]
Example output:   [[bat], [tan, nat], [eat, tea, ate]]

Additional 1 in:  []
Additional 1 out: [[]]

Edge input:       []
Edge output:      []
```

Figure 4: Problem 4 execution screenshot

2. Design the Solution

Sort intervals by start, then check adjacent pairs. If any next start is strictly less than the previous end, there is a conflict.

3. Pseudocode

```
1 function canAttendAll(intervals):
2     if intervals.size <= 1: return true
3     sort intervals by start ascending
4     for i from 1 to intervals.size - 1:
5         if intervals[i].start < intervals[i-1].end:
6             return false
7     return true
```

4. Correctness

After sorting by start, any conflict must occur between adjacent meetings. Touching endpoints have $\text{start} == \text{end}$ and are not counted because the check is strict.

5. Complexity Analysis

- Time: $O(n \log n)$.
- Space: $O(1)$ or $O(\log n)$.

6. C++ Implementation

```
1 bool canAttendAll(vector<vector<int>> intervals) {
2     if (intervals.size() <= 1) return true;
3     sort(intervals.begin(), intervals.end(),
4         [](const vector<int>& a, const vector<int>& b) {
5             return a[0] < b[0];
6         });
7     for (size_t i = 1; i < intervals.size(); i++) {
```

```

8         if (intervals[i][0] < intervals[i - 1][1]) return false;
9     }
10    return true;
11 }

```

7. Test Cases

Example: `[[0,30],[5,10],[15,20]]`, output false

Additional 1: `[[7,10],[2,4]]`, output true

Edge: `[]`, output true

```

--- Problem 5: Can Attend All Meetings ---
Example input:  [[0,30], [5,10], [15,20]]
Example output: false

Additional 1 in:  [[7,10], [2,4]]
Additional 1 out: true

Edge input:      []
Edge output:     true

```

Figure 5: Problem 5 execution screenshot

Problem 6.

1. Understand the Problem

Goal: Given points (x, y) and integer k , return the k closest points to the origin.

Input: `vector<vector<int>>` points, integer k .

Output: `vector<vector<int>>` of k closest points.

Edge Cases: $k = 0$; $k \geq n$; ties.

2. Design the Solution

Sorting: compute squared distance, sort, take first k .

Heap: max heap of size k keyed by distance.

3. Pseudocode

```

1 function kClosestSort(points, k):
2     sort points by squared distance ascending

```

```

3     return first k points
4
5 function kClosestHeap(points, k):
6     maxHeap = empty max heap by distance
7     for each point in points:
8         push point into maxHeap
9         if maxHeap.size > k: pop maxHeap
10    return all points in maxHeap

```

4. Correctness

Squared distance preserves the ordering of Euclidean distance. Heap keeps the k smallest distances seen so far; by the end, it holds the global k closest points.

5. Complexity Analysis

- Sort: $O(n \log n)$ time, $O(n)$ space.
- Heap: $O(n \log k)$ time, $O(k)$ space.

6. C++ Implementation

```

1 vector<vector<int>> kClosestSort(vector<vector<int>> points, int k) {
2     sort(points.begin(), points.end(), [](const vector<int>& a, const
3         vector<int>& b) {
4         return a[0]*a[0] + a[1]*a[1] < b[0]*b[0] + b[1]*b[1];
5     });
6     if (k > (int)points.size()) k = (int)points.size();
7     return vector<vector<int>>(points.begin(), points.begin() + k);
8 }
9
10 vector<vector<int>> kClosestHeap(vector<vector<int>> points, int k) {
11     if (k <= 0) return {};
12     auto cmp = [](const vector<int>& a, const vector<int>& b) {
13         return a[0]*a[0] + a[1]*a[1] < b[0]*b[0] + b[1]*b[1];
14     };
15     priority_queue<vector<int>, vector<vector<int>>, decltype(cmp)>
16         maxHeap(cmp);
17     for (const auto& p : points) {
18         maxHeap.push(p);
19         if ((int)maxHeap.size() > k) maxHeap.pop();
20     }
21     vector<vector<int>> result;
22     while (!maxHeap.empty()) { result.push_back(maxHeap.top()); maxHeap.
    pop(); }
    return result;
}

```

7. Test Cases

Example: $[[1,3], [-2,2]]$, $k = 1$, output $[-2,2]$

Additional 1: $[[3,3], [5,-1], [-2,4]]$, $k = 2$

Edge: [], $k = 0$, output []

```
--- Problem 6: K Closest Points ---
Example input:  [[1,3], [-2,2]], k = 1
Sort out:      [[-2,2]]
Heap out:      [[-2,2]]

Additional 1 in: [[3,3], [5,-1], [-2,4]], k = 2
Sort out:      [[3,3], [-2,4]]
Heap out:      [[-2,4], [3,3]]

Edge input:    [], k = 0
Sort out:      []
Heap out:      []
```

Figure 6: Problem 6 execution screenshot

Problem 7.

1. Understand the Problem

Goal: Given an integer array and k , return the k most frequent values. Ties break by smaller value first.

Input: `vector<int> nums`, integer k .

Output: `vector<int>`.

Edge Cases: empty; $k = 0$; all equal frequencies; ties.

2. Design the Solution

Hash map for frequencies; vector of pairs; sort by frequency desc, value asc; take first k .

3. Pseudocode

```
1 function topKFrequent(nums, k):
2     freqMap = empty hash map
3     for x in nums: freqMap[x]++
4     items = list of (value, frequency) from freqMap
5     sort items by:
6         if frequency different: frequency descending
7         else: value ascending
8     return first k values
```

4. Correctness

Frequencies are exact. The comparator matches the required tie-break rule. Taking the first k entries yields the answer.

5. Complexity Analysis

- Time: $O(n + m \log m)$, $m = \text{distinct values}$.
- Space: $O(m)$.

6. C++ Implementation

```
1 vector<int> topKFrequent(vector<int> nums, int k) {
2     unordered_map<int, int> freq;
3     for (int x : nums) freq[x]++;
4     vector<pair<int, int>> items;
5     for (auto& p : freq) items.push_back({p.first, p.second});
6     sort(items.begin(), items.end(), [](const pair<int, int>& a, const
7         pair<int, int>& b) {
8         if (a.second != b.second) return a.second > b.second;
9         return a.first < b.first;
10    });
11    vector<int> result;
12    for (int i = 0; i < k && i < (int)items.size(); i++) result.push_back(
13        items[i].first);
14    return result;
15 }
```

7. Test Cases

Example: nums [1,1,1,2,2,3], $k = 2$, output [1,2]

Additional 1: nums [1], $k = 1$, output [1]

Edge: nums [4,4,4,4], $k = 1$, output [4]

Problem 8.

1. Understand the Problem

Goal: Given k sorted arrays, merge them into one sorted array without re-sorting the whole output.

Input: vector<vector<int>> of sorted arrays.

Output: a single sorted vector<int>.

Edge Cases: empty list; arrays containing empty arrays; single array; all arrays empty.

2. Design the Solution

Min-heap storing (value, arrayIndex, elementIndex). Push first element of every non-empty array. Pop min, append, push next from same array.

```

--- Problem 7: Top K Frequent Elements ---
Example input:      [1, 1, 1, 2, 2, 3], k = 2
Example output:     [1, 2]

Additional 1 in:    [1], k = 1
Additional 1 out:   [1]

Edge input:         [4, 4, 4, 4], k = 1
Edge output:        [4]

```

Figure 7: Problem 7 execution screenshot

3. Pseudocode

```

1 function mergeKSorted(arrays):
2     result = empty list
3     minHeap = empty min heap of (value, arrayIndex, elementIndex)
4     for i from 0 to arrays.size - 1:
5         if arrays[i] not empty:
6             push (arrays[i][0], i, 0)
7     while minHeap not empty:
8         (value, i, j) = pop minHeap
9         result.add(value)
10        if j + 1 < arrays[i].size:
11            push (arrays[i][j+1], i, j+1)
12    return result

```

4. Correctness

Heap holds the smallest unprocessed element of each array, so its minimum is the global next smallest. Replenishment keeps the invariant, so output is fully sorted.

5. Complexity Analysis

- Time: $O(N \log k)$.
- Space: $O(k)$ heap, $O(N)$ result.

6. C++ Implementation

```

1 vector<int> mergeKSorted(vector<vector<int>> arrays) {
2     struct Node {
3         int value; int arrayIndex; int elementIndex;
4         bool operator>(const Node& other) const { return value > other.
           value; }

```

```

5     };
6     priority_queue<Node, vector<Node>, greater<Node>> minHeap;
7     for (int i = 0; i < (int)arrays.size(); i++) {
8         if (!arrays[i].empty()) minHeap.push({arrays[i][0], i, 0});
9     }
10    vector<int> result;
11    while (!minHeap.empty()) {
12        Node node = minHeap.top(); minHeap.pop();
13        result.push_back(node.value);
14        int nextIndex = node.elementIndex + 1;
15        if (nextIndex < (int)arrays[node.arrayIndex].size()) {
16            minHeap.push({arrays[node.arrayIndex][nextIndex], node.
17                           arrayIndex, nextIndex});
18        }
19    }
20    return result;
}

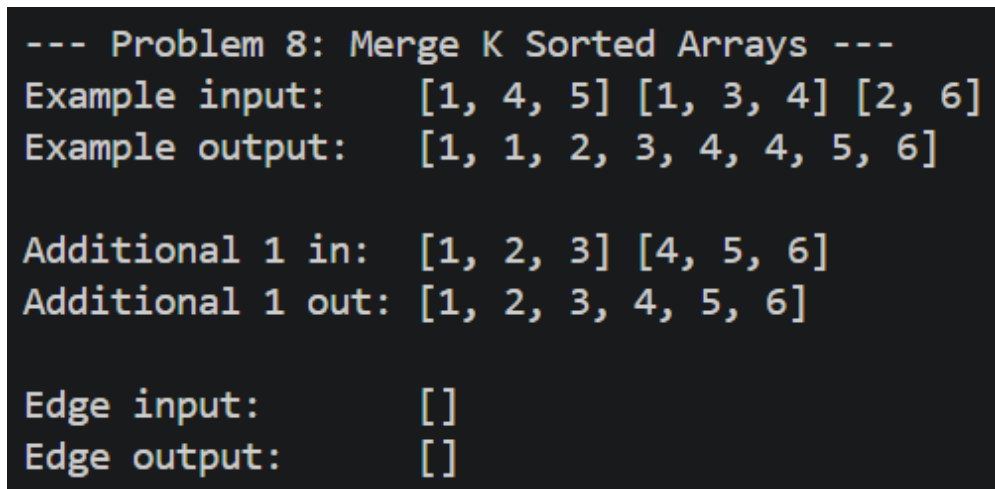
```

7. Test Cases

Example: `[[1,4,5],[1,3,4],[2,6]]`, output `[1,1,2,3,4,4,5,6]`

Additional 1: `[[1,2,3],[4,5,6]]`, output `[1,2,3,4,5,6]`

Edge: `[]`, output `[]`



```

--- Problem 8: Merge K Sorted Arrays ---
Example input:      [1, 4, 5] [1, 3, 4] [2, 6]
Example output:     [1, 1, 2, 3, 4, 4, 5, 6]

Additional 1 in:    [1, 2, 3] [4, 5, 6]
Additional 1 out:   [1, 2, 3, 4, 5, 6]

Edge input:         []
Edge output:        []

```

Figure 8: Problem 8 execution screenshot

Problem 9.

1. Understand the Problem

Goal: Given an unsorted integer array and k , return the k th largest element. Duplicates count as separate positions.

Input: `vector<int> nums`, integer k .

Output: integer.

Edge Cases: empty; $k = 1$; $k = n$; duplicates.

2. Design the Solution

Quickselect: partition around a pivot and recurse into the side containing the target index $n - k$ (ascending).

3. Pseudocode

```
1 function findKthLargest(nums, k):
2     target = nums.size() - k
3     left = 0; right = nums.size() - 1
4     while true:
5         p = partition(nums, left, right)
6         if p == target: return nums[p]
7         else if p < target: left = p + 1
8         else: right = p - 1
9
10 function partition(nums, left, right):
11     pivot = nums[right]
12     i = left
13     for j from left to right - 1:
14         if nums[j] <= pivot: swap(nums[i], nums[j]); i++
15     swap(nums[i], nums[right])
16     return i
```

4. Correctness

After partition, elements left of pivot are $\leq$ pivot, right are $\geq$ pivot. If pivot index equals target, that position is the answer. Otherwise the search narrows into the correct side. The loop terminates when found.

5. Complexity Analysis

- Average time: $O(n)$.
- Worst-case time: $O(n^2)$.
- Space: $O(1)$.

6. C++ Implementation

```
1 int partition(vector<int>& nums, int left, int right) {
2     int pivot = nums[right];
3     int i = left;
4     for (int j = left; j < right; j++) {
5         if (nums[j] <= pivot) { swap(nums[i], nums[j]); i++; }
6     }
7     swap(nums[i], nums[right]);
8     return i;
```

```

9  }
10
11 int findKthLargest(vector<int> nums, int k) {
12     int target = (int)nums.size() - k;
13     int left = 0, right = (int)nums.size() - 1;
14     while (true) {
15         int p = partition(nums, left, right);
16         if (p == target) return nums[p];
17         else if (p < target) left = p + 1;
18         else right = p - 1;
19     }
20 }

```

7. Test Cases

Example: nums [3,2,1,5,6,4], $k = 2$, output 5

Additional 1: nums [3,2,3,1,2,4,5,5,6], $k = 4$, output 4

Edge: nums [1], $k = 1$, output 1

```

--- Problem 9: Kth Largest Element ---
Example input:    [3, 2, 1, 5, 6, 4], k = 2
Example output:   5

Additional 1 in:  [3, 2, 3, 1, 2, 4, 5, 5, 6], k = 4
Additional 1 out: 4

Edge input:       [1], k = 1
Edge output:      1

```

Figure 9: Problem 9 execution screenshot

Problem 10.

1. Understand the Problem

Goal: Count pairs (i, j) with $i < j$ and $values[i] > values[j]$.

Input: vector<int>.

Output: long long (may exceed 32-bit).

Constraints: Main solution must be $O(n \log n)$ using modified Merge Sort.

Edge Cases: empty; single element; sorted; reverse sorted; duplicates (equal values do not count).

2. Design the Solution

Modify Merge Sort: during merge, if left element $>$ right element, add $mid - i + 1$ to the count.

3. Pseudocode

```
1 function countInversions(values):
2     if values.size <= 1: return 0
3     temp = array of same size
4     return mergeSortCount(values, temp, 0, values.size - 1)
5
6 function mergeSortCount(arr, temp, left, right):
7     if left >= right: return 0
8     mid = (left + right) / 2
9     count = 0
10    count += mergeSortCount(arr, temp, left, mid)
11    count += mergeSortCount(arr, temp, mid + 1, right)
12    count += mergeAndCount(arr, temp, left, mid, right)
13    return count
14
15 function mergeAndCount(arr, temp, left, mid, right):
16    i = left; j = mid + 1; k = left; count = 0
17    while i <= mid and j <= right:
18        if arr[i] <= arr[j]: temp[k++] = arr[i++]
19        else:
20            temp[k++] = arr[j++]
21            count += (mid - i + 1)
22    while i <= mid: temp[k++] = arr[i++]
23    while j <= right: temp[k++] = arr[j++]
24    for p from left to right: arr[p] = temp[p]
25    return count
```

4. Correctness

During merge, if $\text{arr}[i] > \text{arr}[j]$, every element from i to mid is also greater than $\text{arr}[j]$, adding $\text{mid} - i + 1$ in one step. Using $\leq$ means equal values never count, matching the strict inequality requirement.

5. Complexity Analysis

- Time: $O(n \log n)$ all cases.
- Space: $O(n)$ for temporary array.

6. C++ Implementation

```
1 long long mergeAndCount(vector<int>& arr, vector<int>& temp, int left, int
   mid, int right) {
2     int i = left, j = mid + 1, k = left;
3     long long count = 0;
4     while (i <= mid && j <= right) {
5         if (arr[i] <= arr[j]) temp[k++] = arr[i++];
6         else { temp[k++] = arr[j++]; count += (mid - i + 1); }
7     }
8     while (i <= mid) temp[k++] = arr[i++];
9     while (j <= right) temp[k++] = arr[j++];
```

```

10     for (int p = left; p <= right; p++) arr[p] = temp[p];
11     return count;
12 }
13
14 long long mergeSortCount(vector<int>& arr, vector<int>& temp, int left,
15     int right) {
16     if (left >= right) return 0;
17     int mid = left + (right - left) / 2;
18     long long count = 0;
19     count += mergeSortCount(arr, temp, left, mid);
20     count += mergeSortCount(arr, temp, mid + 1, right);
21     count += mergeAndCount(arr, temp, left, mid, right);
22     return count;
23 }
24
25 long long countInversions(vector<int> values) {
26     if (values.size() <= 1) return 0;
27     vector<int> temp(values.size());
28     return mergeSortCount(values, temp, 0, (int)values.size() - 1);
29 }

```

7. Test Cases

Example: input [2,4,1,3,5], output 3

Additional 1: input [5,4,3,2,1], output 10

Edge: input [], output 0

```

--- Problem 10: Count Inversions ---
Example input:    [2, 4, 1, 3, 5]
Example output:   3

Additional 1 in:  [5, 4, 3, 2, 1]
Additional 1 out: 10

Edge input:      []
Edge output:     0

```

Figure 10: Problem 10 execution screenshot

Appendix: GitHub Repository

GitHub Repository Link:

https://github.com/RATH-666/EECE7205_Assignment01-Lingtng.Ge